



Diseño de IOT y sistemas embebidos

Laboratorio 1

Introducción a la placa de desarrollo y a VSCode + ESP-IDF

Docentes:

Nicolas Calarco y Pablo Alonso

Estudiantes:

Victoria Amor, Nicolas Meyer y Bruno Moreira

22 de Abril de 2026

1. Introducción

El presente laboratorio tiene como objetivo comenzar a usar la placa de desarrollo ESP32-S2-KALUGA-1 y del entorno de programación ESP-IDF integrado en Visual Studio Code para establecer una primera aproximación práctica al desarrollo de sistemas embebidos. A lo largo del laboratorio se busca adquirir experiencia en la estructura de un proyecto en ESP-IDF, con los archivos de configuración que intervienen en su compilación y con el flujo de trabajo fundamental compuesto por las etapas de creación, compilación, flasheo y monitoreo del firmware.

En una primera instancia, el trabajo se orienta al reconocimiento del entorno de desarrollo, analizando la organización de los archivos generados automáticamente, la función del archivo `sdkconfig` y la información producida durante la compilación, incluyendo el estudio del mapa de memoria y de los archivos auxiliares involucrados en el proceso de flasheo. Posteriormente, el laboratorio se enfoca en realizar una aplicación sencilla sobre la placa, que consiste en controlar el LED RGB integrado. Para esto, se plantea escribir código en C organizándolo modularmente, de forma que por un lado se maneje el funcionamiento del LED y por otro se implementen funciones de tiempo (delays), las cuales luego se llaman desde el programa principal.

2. Introducción al entorno ESP-IDF + VSCode

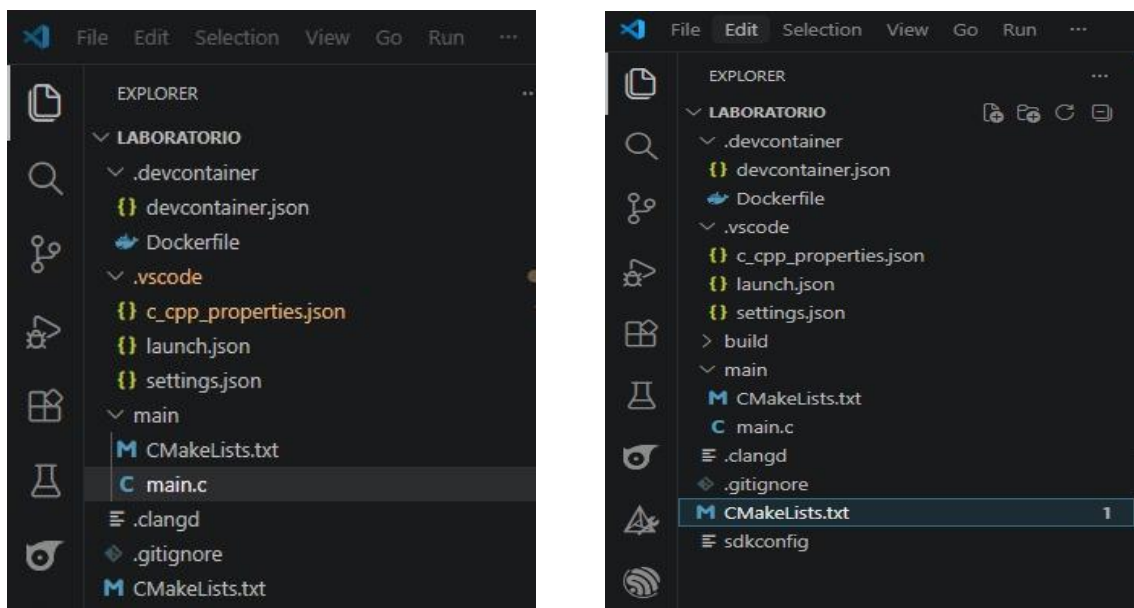
2.1 Creación de un proyecto

Se comienza creando un nuevo proyecto a partir de la plantilla `sample_project`, lo cual genera automáticamente una estructura de carpetas y archivos base necesarios para el desarrollo (figura 1). Entre las carpetas principales se encuentra `.devcontainer/`, junto con los archivos `devcontainer.json` y `Dockerfile` que contienen configuraciones relacionadas con entornos de desarrollo basados en contenedores (Docker), permitiendo definir dependencias y herramientas necesarias para trabajar en ese entorno (Microsoft, 2026). Por otro lado, la carpeta `.vscode/` incluye archivos de configuración del editor, como `c_cpp_properties.json`, que permite configurar el sistema de IntelliSense, definiendo rutas de inclusión, macros del preprocesador y el compilador utilizado para el análisis del código (Microsoft, 2025), `launch.json`, utilizado para la configuración de la depuración (Microsoft, 2023), y `settings.json`, que almacena preferencias del entorno de trabajo (Microsoft, 2026).

Asimismo, la carpeta `main/` contiene el código fuente principal del proyecto, donde el archivo `main.c` corresponde al programa que será ejecutado en la placa, mientras que su archivo `CMakeLists.txt` asociado define cómo se compila dicho código. El otro archivo `CMakeLists.txt` no asociado a `main.c` establece la configuración general de compilación y permite integrar los distintos componentes del sistema (Espressif Systems, s. f.). Además, el archivo `.clangd` sirve para configurar el comportamiento de `clangd`, permitiendo definir parámetros que afectan el análisis del código fuente, como flags de compilación

y diagnósticos, los cuales se utilizan para detectar errores, advertencias y sugerir correcciones automáticas durante la edición del código (clangd, s. f.). Por último, el archivo `.gitignore` indica qué archivos o carpetas deben ser ignorados en el control de versiones (Git, s. f.).

Luego de crear el proyecto, se realiza una compilación inicial sin modificar el código. Como resultado, se genera una nueva carpeta denominada `build/`, la cual contiene todos los archivos intermedios y finales del proceso de compilación. Es decir, archivos binarios, objetos y configuraciones necesarias para generar el firmware. Además, se genera el archivo `sdkconfig`, el cual contiene la configuración del proyecto, incluyendo distintos parámetros del sistema que se utilizan durante la compilación (figura 2) (Espressif Systems, s.f.).



Figuras 1 y 2: Estructura del proyecto al crearlo y archivos generados luego de la compilación respectivamente.

En la terminal se muestra información sobre este proceso de compilación. Se puede observar la ejecución de herramientas como CMake y Ninja, que se encargan de configurar y compilar el proyecto. También se muestra información sobre el compilador utilizado y los distintos componentes del sistema que forman parte del programa. Además, se indica que se está utilizando el archivo `sdkconfig` para la configuración del proyecto y se presenta un resumen de dicha configuración. Finalmente, se informa la generación de archivos binarios como el `bootloader` y el archivo principal del proyecto, junto con su tamaño, y se muestran comandos que pueden utilizarse para cargar el programa en la placa.

2.2 Parámetros de configuración (sdkconfig)

El archivo `sdkconfig` se genera automáticamente durante la compilación y guarda las configuraciones del programa que se está desarrollando para la placa. En este archivo se definen distintas opciones que controlan cómo funciona el programa, incluyendo configuraciones del microcontrolador, la activación de

distintas funciones del sistema y parámetros que determinan cómo se genera el código al compilar (Espressif Systems, s. f.).

Mirando las primeras líneas del archivo `sdkconfig`, se observan configuraciones relacionadas con las capacidades del sistema, como el soporte para distintos periféricos del microcontrolador como el ADC y DAC. También aparecen opciones vinculadas al sistema de timers y manejo de GPIO, lo que indica qué recursos de hardware están disponibles y habilitados para su uso.

Estos parámetros pueden modificarse mediante una interfaz gráfica desde VSCode (figura 3) o utilizando el comando `idf.py menuconfig` (figura 4 y 5). Cada vez que se realiza un cambio, el archivo `sdkconfig` se actualiza y se guarda una copia anterior en `sdkconfig.old`, lo que permite comparar configuraciones y restaurar valores previos.

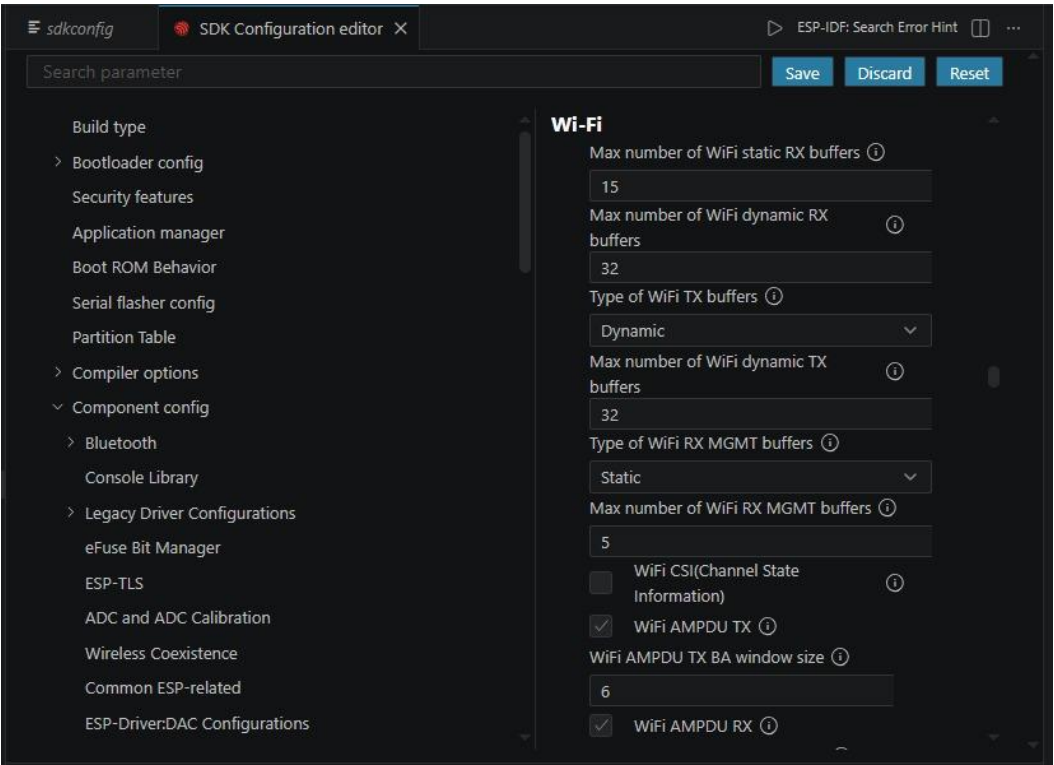


Figura 3: Interfaz gráfica de VSCode para la configuración del archivo `sdkconfig`.

```
(Top)
Espressif IoT Development Framework Configuration
Build type --->
Bootloader config --->
↓↓↓↓↓↓↓↓↓↓↓↓↓↓
[Space/Enter] Toggle/enter  [ESC] Leave menu          [S] Save
[O] Load                  [?] Symbol info          [/] Jump to symbol
[F] Toggle show-help mode  [C] Toggle show-name mode [A] Toggle show-all mode
[Q] Quit (prompts for save) [D] Save minimal config (advanced) [R] Reset to default value
```

```
(Top) → Component config → Wi-Fi
Espressif IoT Development Framework Configuration
(15) Max number of WiFi static RX buffers
(32) Max number of WiFi dynamic RX buffers (default value)
Type of WiFi TX buffers (Dynamic) (default value) --->
(32) Max number of WiFi dynamic TX buffers (default value)
Type of WiFi RX MGMT buffers (Static) (default value) --->
(5) Max number of WiFi RX MGMT buffers (default value)
[ ] WiFi CSI(Channel State Information)
[*] WiFi AMPDU TX (default value)
(6) WiFi AMPDU TX Buffer Size (default value)
[*] WiFi AMPDU RX (default value)
(6) WiFi AMPDU RX Buffer Size (default value)
[*] WiFi NVS flash (default value)
(752) Max length of WiFi SoftAP Beacon (default value)
(32) WiFi mgmt short buffer number (default value)
↓↓↓↓↓↓↓↓↓↓↓↓↓↓
[Space/Enter] Toggle/enter [ESC] Leave menu [S] Save
[O] Load [?] Symbol info [/] Jump to symbol
[F] Toggle show-help mode [C] Toggle show-name mode [A] Toggle show-all mode
[Q] Quit (prompts for save) [D] Save minimal config (advanced) [R] Reset to default value
```

Figuras 4 y 5: Interfaz de configuración del proyecto utilizando idf.py menuconfig en la terminal.

En particular, se modificó el parámetro WiFi static RX buffers, cambiando su valor de 10 a 15, el cual se encuentra dentro de la sección Component config → Wi-Fi. Al guardar esta modificación, se observó que el archivo sdkconfig se actualiza con el nuevo valor (CONFIG_ESP_WIFI_STATIC_RX_BUFFER_NUM=15), mientras que sdkconfig.old conserva la configuración anterior (figuras 6 y 7).

```
C main.c  sdkconfig  Laboratorio.map
sdkconfig
1855 #
1856 # Wi-Fi
1857 #
1858 # default:
1859 CONFIG_ESP_WIFI_ENABLED=y
1860 CONFIG_ESP_WIFI_STATIC_RX_BUFFER_NUM=15
1861 # default:
```

```
C main.c  sdkconfig.old  Laboratorio.map
sdkconfig.old
1855 #
1856 # Wi-Fi
1857 #
1858 # default:
1859 CONFIG_ESP_WIFI_ENABLED=y
1860 # default:
1861 CONFIG_ESP_WIFI_STATIC_RX_BUFFER_NUM=10
```

Figuras 6 y 7: Archivo sdkconfig modificado y su versión anterior en sdkconfig.old respectivamente.

2.3 Compilación y análisis del mapa de memoria

En esta sección se agregan definiciones y variables al programa para analizar cómo se almacenan en memoria.

Luego de compilar el proyecto con estas modificaciones, se generan distintos archivos en la carpeta `build/`, los cuales contienen información asociada tanto al proceso de compilación como a la generación del firmware. Entre ellos están `flasher_args.json` y `project_description.json`.

El archivo `flasher_args.json` contiene la información necesaria para cargar el firmware en la placa. En las primeras líneas se observa la configuración de flasheo, por ejemplo, el modo de memoria flash (`dio`), el tamaño de esta (2MB) y la frecuencia (80m). También se indican los archivos binarios que deben escribirse y la dirección de memoria donde se carga cada uno: el bootloader en `0x1000`, la tabla de particiones en `0x8000` y la aplicación principal en `0x10000`. Además, se incluyen parámetros adicionales para `esptool`, como el tipo de chip (`esp32s2`) y el comportamiento antes y después del flasheo (`default-reset` y `hard-reset`). Básicamente, este archivo sirve como guía para que la herramienta de flasheo (`esptool`) sepa qué archivos escribir, en qué direcciones hacerlo y con qué configuración de memoria.

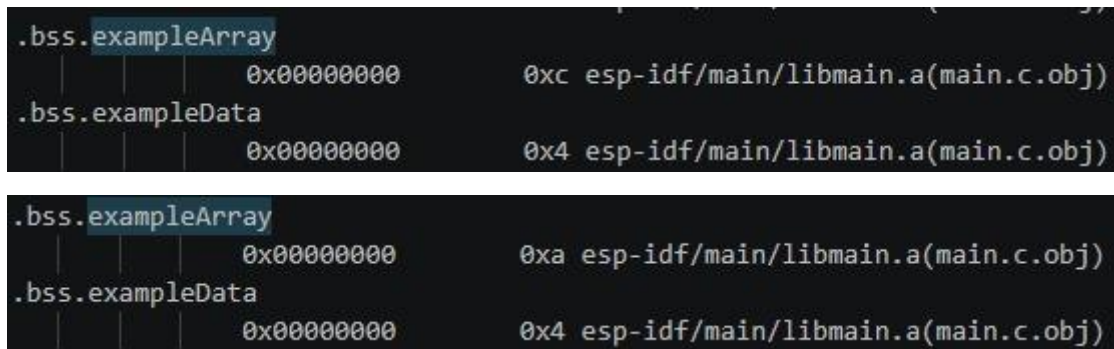
Por otro lado, el archivo `project_description.json` contiene información general del programa compilado y del entorno usado para generarlo. En las primeras líneas aparecen datos como el nombre del programa (`Laboratorio`), la versión, la ruta del proyecto, la ruta de la carpeta `build`, el archivo de configuración utilizado (`sdkconfig`), el archivo ejecutable generado (`Laboratorio.elf`), el binario final (`Laboratorio.bin`), el tipo de compilación (`flash_app`) y el microcontrolador objetivo (`esp32s2`). También se indica la velocidad del monitor serie (`115200`) y el compilador utilizado. Este archivo no es el que escribe directamente en la memoria, pero sirve para describir cómo fue construido el programa y con qué configuración (Espressif Systems, s. f.).

Por último, se analiza el archivo `.map`, el cual permite observar cómo se distribuyen las variables en la memoria, indicando tanto su dirección como el espacio que ocupan (Microchip, s. f.).

Específicamente para este caso, se observa que la variable `exampleData`, al ser de tipo `int`, ocupa `0x4` bytes, es decir, 4 bytes. Por otro lado, el array `exampleArray`, definido como `char exampleArray[ARRAY_SIZE]` (con `ARRAY_SIZE = 12`), ocupa `0xc` bytes, es decir, 12 bytes. Ambas variables aparecen dentro de la sección `.bss`, que corresponde a variables globales no inicializadas (GNU Binutils, s. f.) y figuran en la dirección `0x00000000`.

Al modificar el valor de `ARRAY_SIZE` de 12 a 10, el tamaño del array `exampleArray` disminuye de `0xc` bytes (12 bytes) a `0xa` bytes (10 bytes). En cuanto a la dirección, la variable sigue apareciendo en `0x00000000` porque se

mantiene ubicada al inicio de la sección .bss. Esto indica que no cambió su posición relativa dentro de la sección, sino únicamente el espacio de memoria que ocupa (figuras 8 y 9).



The image contains two screenshots of a memory layout, likely from a linker script or a debugger. Both screenshots show a table with columns for variable names, addresses, and offsets. The first screenshot shows the state before a modification, and the second shows the state after. In both, the variable `.bss.exampleArray` is at address `0x00000000` with an offset of `0xc` (before) or `0xa` (after). The variable `.bss.exampleData` is at address `0x00000000` with an offset of `0x4` in both states.

Variable	Address	Offset
<code>.bss.exampleArray</code>	<code>0x00000000</code>	<code>0xc esp-idf/main/libmain.a(main.c.obj)</code>
<code>.bss.exampleData</code>	<code>0x00000000</code>	<code>0x4 esp-idf/main/libmain.a(main.c.obj)</code>

Variable	Address	Offset
<code>.bss.exampleArray</code>	<code>0x00000000</code>	<code>0xa esp-idf/main/libmain.a(main.c.obj)</code>
<code>.bss.exampleData</code>	<code>0x00000000</code>	<code>0x4 esp-idf/main/libmain.a(main.c.obj)</code>

Figuras 8 y 9: Distribución de las variables en la memoria antes y después de la modificación respectivamente.

2.4 Flasheo y monitoreo del programa

Una vez compilado el proyecto, se procede a cargar el firmware en la placa mediante el proceso de flasheo. Para esto, es necesario conectar la placa a la computadora, seleccionar el puerto correspondiente (COM8 en este caso) y ejecutar la acción de flash desde VS Code.

Una vez iniciado el proceso, la terminal muestra que se establece la comunicación con el microcontrolador ESP32-S2, indicando información como el tipo de chip, sus características y la frecuencia del cristal.

Luego, mediante la herramienta esptool, se lleva a cabo la escritura del firmware en la memoria flash. En la salida de la terminal se observa que se transfieren distintos archivos binarios a direcciones específicas de memoria. En primer lugar, se escribe el bootloader en la dirección `0x00001000`, que participa en el proceso de arranque del sistema, cargando desde la memoria flash tanto la tabla de particiones como la aplicación principal (Espressif Systems, s. f.). A continuación, se carga la recién mencionada `partition table` en la dirección `0x00008000`, la cual define las distintas particiones del sistema indicando su tipo y la dirección en memoria flash donde se ubican (Espressif Systems, s. f.). Finalmente, se escribe la aplicación principal (`Laboratorio.bin`) en la dirección `0x00010000`, que corresponde al programa desarrollado (app).

Durante cada una de estas etapas, la terminal indica el tamaño de los datos escritos, el rango de memoria que se borra previamente en la flash y la velocidad de transferencia. Al finalizar el proceso, se ejecuta un reinicio automático de la placa ("Hard resetting via RTS pin"), lo que permite que el programa cargado comience a ejecutarse.

Posteriormente, se utiliza el monitor para observar la salida del programa en ejecución. En la terminal se muestran automáticamente distintos mensajes del sistema, sin haber agregado más código. Entre los mensajes se observa información del arranque del microcontrolador, como el reinicio ("POWERON"), la

carga de los distintos segmentos del programa (load:0x3ffe5110,len:0x1624) y la ejecución del bootloader. Posteriormente, se muestra información del sistema, incluyendo la versión de ESP-IDF utilizada, la configuración de la memoria, la tabla de particiones y los datos del programa cargado, como su nombre ("Laboratorio"), versión y tiempo de compilación. Finalmente, aparecen mensajes como "Calling app_main()" y "Returned from app_main()", indicando que la función principal del programa fue ejecutada.

Aclarar que los mensajes observados en la terminal no provienen del código del usuario, sino que son generados automáticamente por el sistema del ESP32 durante su ejecución. En particular, se originan en el sistema de logging del ESP-IDF, cuyo nivel de detalle se configura mediante el archivo sdkconfig (Espressif Systems, s. f.).

Para finalizar, se agregó un printf dentro de la función principal del programa (app_main), luego se volvió a compilar y flashear el programa. En el monitor se pudo observar el mensaje agregado en la salida de la terminal, lo que permitió verificar que el programa desarrollado se estaba ejecutando correctamente en la placa (figura 10).



```
I (216) heap_init: At 3FF9E000 len 00001FE8 (7 KiB): RTCRAM
I (222) spi_flash: detected chip: gd
I (225) spi_flash: flash io: dio
W (227) spi_flash: Detected size(4096k) larger than the size in the binary image header(2048k). Using the size in the binary image header.
I (240) main_task: Started on CPU0
I (240) main_task: Calling app_main()
Sistemas Embebidos y IOT 2026 prueba
I (240) main_task: Returned from app_main()
```

Figura 10: Mensaje impreso en la terminal tras la ejecución del programa en la placa.

3. "Hola Mundo" embebido: parpadeo de LED RGB

3.1 LED RGB WS2812C

En esta sección se trabajó con el LED RGB WS2812C, cuyo funcionamiento se basa en un protocolo de comunicación en serie del tipo NZR (Non-Return-to-Zero). Luego de un power-on reset, el componente recibe por el pin DIN una secuencia continua de datos proveniente del microcontrolador (Figuras 11 y 12).

Cada LED toma los primeros 24 bits de los datos recibidos y los almacena en un latch interno, mientras que los bits restantes son regenerados mediante un circuito interno de reshaping (reconstrucción y amplificación de señal), y retransmitidos por el pin DO hacia el siguiente. De esta forma, en cada etapa de la transmisión, el flujo de datos se reduce en 24 bits por LED (Figura 12).

Gracias a la tecnología de auto-reshaping, la señal se reconstruye en cada nodo, lo que permite encadenar un gran número de píxeles sin degradación significativa, siendo la limitación principal la velocidad de transmisión de la señal.

Cada bit transmitido representa las componentes de cada color ([G (verde)][R (rojo)][B (azul)]), con 8 bits para cada componente, permitiendo así formar el resto de colores ya que son primarios .La transmisión de estos bits se realiza de forma secuencial, comenzando por el bit más significativo de cada componente (Figura 12).

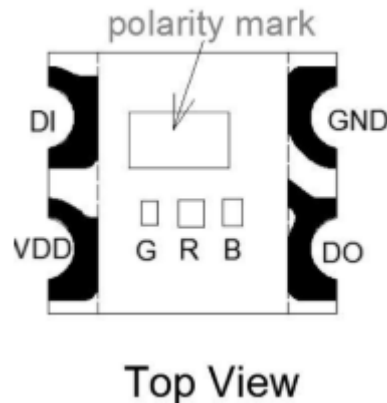
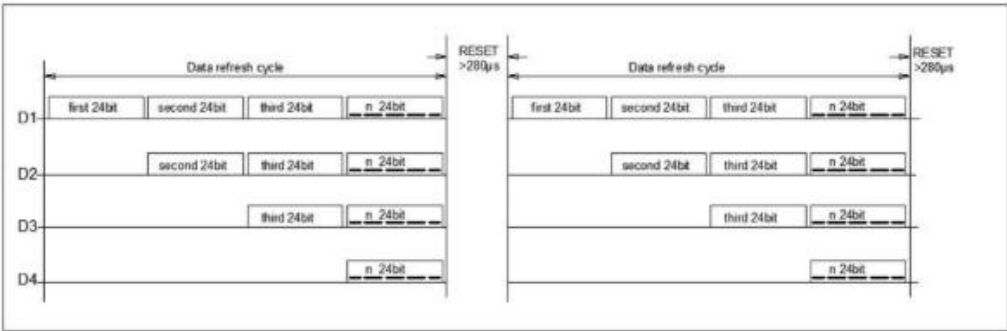


Figura 11: LED WS2812C (visto desde arriba).

Data Transmission Method



Note: The data of D1 is send by MCU, and D2, D3, D4 through pixel internal reshaping amplification to transmit.

Composition of 24bit Data

G7	G6	G5	G4	G3	G2	G1	G0	R7	R6	R5	R4	R3	R2	R1	R0	B7	B6	B5	B4	B3	B2	B1	B0
----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----

Note: Data transmit in order of GRB, high bit data at first.

Figura 12: Método de transmisión y composición de los 24 bits de datos por LED.

El correcto funcionamiento del LED depende de respetar estrictamente los tiempos críticos de la señal, ya que la codificación de cada bit no se realiza mediante niveles lógicos directos, sino mediante la duración de los pulsos en nivel alto (H) y nivel bajo (L). En este contexto, un bit lógico '0' y un bit lógico '1' no se distinguen por el valor de la señal en sí, sino por cuánto tiempo permanece la señal en nivel alto dentro de un período fijo.

En particular, un bit '0' se representa mediante un pulso en nivel alto de corta duración seguido de un pulso en nivel bajo más largo, mientras que un bit '1' se representa mediante un pulso en nivel alto de mayor duración y un pulso en nivel bajo más corto. A pesar de estas diferencias, el período total de cada bit (suma de tiempo en alto y en bajo) se mantiene aproximadamente constante, con un valor cercano a 1,25 μ s, lo que corresponde a una velocidad de transmisión de aproximadamente 800 kbps.

Los parámetros indicados en la figura 13 corresponden a estos intervalos: los tiempos T0H y T1H representan la duración del nivel alto para los bits '0' y '1', respectivamente, mientras que T0L y T1L corresponden a los tiempos en nivel bajo.

Además, una vez finalizada la transmisión de la secuencia completa de bits, la línea de datos debe quedarse en nivel bajo durante un tiempo mayor a 280 μ s. En este período el LED interpreta que la transmisión ha finalizado y procede a actualizar el color mostrado con los datos almacenados previamente.

Data Transfer Time

T0H	0 code, high voltage time	220ns~380ns
T1H	1 code, high voltage time	580ns~1 μ s
T0L	0 code, low voltage time	580ns~1 μ s
T1L	1 code, low voltage time	580ns~1 μ s
RES	Frame unit, low voltage time	>280 μ s

Figura 13: Tiempos de transmisión de datos del LED WS2812C.

El proceso de control del LED puede describirse mediante el siguiente pseudocódigo, el cual representa la generación y transmisión de la secuencia de datos según el protocolo NZR.

```
Power_On_Reset()

// Definir colores en formato GRB, 24 bits por LED
led_1 = 11111111 11111111 11111111 // blanco
led_2 = 00000000 00000000 00000000 // apagado

// Construir la trama completa
data = led_1 + led_2

// Enviar la secuencia bit a bit por DIN
for bit in data:
    send_bit_NZR(bit)

// Tiempo de reset para actualizar los LEDs
wait(>280  $\mu$ s)
```

3.2 Librería de manejo del LED embebido

Para el manejo de la led, se partió desde una librería brindada por los profesores, compuesta de `led_strip_rmt_ws2812.c` y `led_strip.h`. Sin embargo, esta librería estaba desactualizada para la versión del entorno utilizada, por lo que hubo que modificarla. La librería original utilizaba `#include "driver/rmt.h"`, la cual actualmente se reemplazó por `#include "driver/rmt_tx.h"` e `#include "driver/rmt_encoder.h"`. La principal diferencia consiste en que, con el driver original, los canales debían ser definidos manualmente, mientras que con el driver actual los canales son creados y definidos dinámicamente con el uso de handles (`rmt_channel_handle_t`) (Espressif Systems ,s.f.).

Como consecuencia de estos cambios, se tuvieron que modificar algunas de las funciones de la librería. Se modificó la función de inicialización del led, por la función `led_strip_new_rmt_ws`, la cual utiliza internamente el nuevo driver (Figura 14). Se modificó también la estructura de configuración `led_strip_config_t` para que cumpla con los requisitos de la implementación actual, en particular en la asignación del campo `dev`, el cual se utiliza para almacenar el handle del canal RMT. A su vez, se eliminaron las dependencias del driver antiguo, como la selección manual de canales o algunas estructuras obsoletas con el nuevo driver.

```
led_strip_t *led_strip_new_rmt_ws2812(const led_strip_config_t *config)
{
    led_strip_t *ret = NULL;
    STRIP_CHECK(config, "configuration can't be null", err, NULL);

    // 24 bits per led -> 3 bytes por LED
    uint32_t ws2812_size = sizeof(ws2812_t) + config->max_leds * 3;
    ws2812_t *ws2812 = calloc(1, ws2812_size);
    STRIP_CHECK(ws2812, "request memory for ws2812 failed", err, NULL);

    ws2812->rmt_channel = (rmt_channel_handle_t)config->dev;
    ws2812->strip_len = config->max_leds;

    ws2812->parent.set_pixel = ws2812_set_pixel;
    ws2812->parent.refresh = ws2812_refresh;
    ws2812->parent.clear = ws2812_clear;
    ws2812->parent.del = ws2812_del;

    return &ws2812->parent;
err:
    return ret;
}
```

Figura 14: Implementación de la función `led_strip_new_rmt_ws2812` adaptada al nuevo driver RMT.

```
led_strip_config_t strip_config = {
    .max_leds = KALUGA_RGB_LED_NUMBER,
    .dev = (led_strip_dev_t)channel,
};
```

Figura 15: Estructura de configuración `led_strip_config_t` utilizada para la inicialización del LED.

Por último, se modificaron las funciones de control como `set_pixel`, `refresh` y `clear` para que coincidan con la librería actual y utilicen `esp_err_t`, lo cual asegura compatibilidad con el manejo de errores del sistema (Figuras 16, 17 y 18).

```
static esp_err_t ws2812_set_pixel(led_strip_t *strip, uint32_t index, uint32_t red, uint32_t green, uint32_t blue)
{
    esp_err_t ret = ESP_OK;
    ws2812_t *ws2812 = __containerof(strip, ws2812_t, parent);
    STRIP_CHECK(index < ws2812->strip_len, "index out of the maximum number of leds", err, ESP_ERR_INVALID_ARG);

    uint32_t start = index * 3;
    // GRB order
    ws2812->buffer[start + 0] = green & 0xFF;
    ws2812->buffer[start + 1] = red & 0xFF;
    ws2812->buffer[start + 2] = blue & 0xFF;

    return ESP_OK;
err:
    return ret;
}
```

```
static esp_err_t ws2812_refresh(led_strip_t *strip, uint32_t timeout_ms)
{
    esp_err_t ret = ESP_OK;
    ws2812_t *ws2812 = __containerof(strip, ws2812_t, parent);

    STRIP_CHECK(ws2812->rmt_channel != NULL, "rmt channel not initialized", err, ESP_ERR_INVALID_STATE);
    STRIP_CHECK(ws2812->encoder != NULL, "encoder not initialized", err, ESP_ERR_INVALID_STATE);

    rmt_transmit_config_t tx_config = {
        .loop_count = 0,
    };

    STRIP_CHECK(
        rmt_transmit(ws2812->rmt_channel,
                     ws2812->encoder,
                     ws2812->buffer,
                     ws2812->strip_len * 3,
                     &tx_config) == ESP_OK,
        "transmit failed", err, ESP_FAIL
    );

    STRIP_CHECK(
        rmt_tx_wait_all_done(ws2812->rmt_channel, timeout_ms) == ESP_OK,
        "wait tx done failed", err, ESP_FAIL
    );

    // Reset/latch time para WS2812, sin usar FreeRTOS
    esp_rom_delay_us(300);

    return ESP_OK;
err:
    return ret;
}
```

```
static esp_err_t ws2812_clear(led_strip_t *strip, uint32_t timeout_ms)
{
    ws2812_t *ws2812 = __containerof(strip, ws2812_t, parent);
    memset(ws2812->buffer, 0, ws2812->strip_len * 3);
    return ws2812_refresh(strip, timeout_ms);
}
```

Figuras 16, 17 y 18. Modificaciones realizadas en las funciones `ws2812_set_pixel`, `ws2812_refresh` y `ws2812_clear`, respectivamente.

Luego, se utilizó esa librería para crear una librería propia, `rgb_led.c` y `rgb_led.h`, las cuales utilizan la librería ya existente y la adaptan para trabajar con una sola led.

En primer lugar, se define un puntero estático al LED, inicializado en NULL, indicando que el dispositivo todavía no fue configurado (Figura 19).

```
static led_strip_t *led = NULL;
```

Figura 19: Definición del puntero estático al LED inicializado en NULL.

La función `rgb_led_init` utiliza la función `led_rgb_init` de la librería `led_strip_rmt_ws2812.c` para inicializar y configurar el LED (Figura 20). En esta función, la condición `if (led != NULL)` verifica si el LED ya fue inicializado previamente. Por otro lado, la llamada a la función `led_rgb_init(&led)` recibe la dirección del puntero porque la función espera un doble puntero (`led_strip_t **strip`). El resultado de esta función se compara con `ESP_OK`, valor definido en ESP-IDF para una acción ejecutada correctamente. En caso de error, el puntero se vuelve a establecer en NULL para evitar el uso de un LED no inicializado.

```
void rgb_led_init(void)
{
    if (led != NULL) {
        return;
    }

    if (led_rgb_init(&led) != ESP_OK) {
        led = NULL;
    }
}
```

Figura 20: Implementación de la función `rgb_led_init`.

La función `rgb_led_set_color` recibe valores numéricos los cuales representan qué color prender y se encarga de actualizar el estado del LED. Inicialmente, verifica que el LED haya sido correctamente inicializado (Figura 21). Luego, utiliza las funciones `set_pixel` y `refresh` de la librería base para asignar y transmitir el color seleccionado. En particular, `set_pixel` utiliza el índice 0, ya que se trabaja con un único LED.

```
void rgb_led_set_color(uint8_t red, uint8_t green, uint8_t blue)
{
    if (led == NULL) {
        return;
    }

    led->set_pixel(led, 0, red, green, blue);
    led->refresh(led, 50);
}
```

Figura 21: Implementación de la función `rgb_led_set_color`.

La función `refresh` toma un timeout de 50 ms para garantizar la correcta transmisión de los datos. Este valor se justifica a partir del datasheet del WS2812C, donde se establece que cada bit se transmite en aproximadamente 1,25 μ s, y que cada LED requiere 24 bits de información, lo que resulta en un tiempo de transmisión cercano a 30 μ s. A esto se suma un tiempo de reset mayor a 280 μ s, necesario para que el LED actualice su estado. En consecuencia, el tiempo total requerido para actualizar un LED es del orden de 300 μ s, es decir, muy inferior a 1 ms. Por lo tanto, utilizar un timeout de 50 ms es más que suficiente, ya que este valor es mucho mayor que el tiempo real de transmisión y actúa como un margen de seguridad suficiente para garantizar que todo está funcionando correctamente.

Finalmente, la función `rgb_led_off` apaga el LED mediante el uso de la función `clear` de la librería `led_strip_rmt_ws2812.c` (Figura 22).

```
void rgb_led_off(void)
{
    if (led == NULL) {
        return;
    }

    led->clear(led, 50);
}
```

Figura 22: Implementación de la función `rgb_led_off`.

El archivo `rgb_led.h` contiene las declaraciones de las funciones implementadas, permitiendo su utilización desde el programa principal (Figura 23).

```
... / rgb_led / include / rgb_led.h / ...
#ifndef RGB_LED_H
#define RGB_LED_H

#include <stdint.h>

// Inicializa el LED
void rgb_led_init(void);

// Setea un color RGB (0-255 cada componente)
void rgb_led_set_color(uint8_t red, uint8_t green, uint8_t blue);

// Apaga el LED
void rgb_led_off(void);

#endif
```

Figura 23: Header de la librería `rgb_led.c`

3.3 Librería de delay

Para esta parte, se implementó una librería propia de retardos basada en funciones provistas por ESP-IDF, definiendo funciones que permiten generar delays en milisegundos y segundos. Para ello hubo que incluir la librería correspondiente mediante `#include "esp_rom_sys.h"`, donde se encuentra definida la función `esp_rom_delay_us`.

La función `delay_ms` utiliza la función interna de ESP-IDF `esp_rom_delay_us`, la cual genera una demora en microsegundos (Figura 24). En este caso, `delay_ms` recibe un valor en milisegundos y lo convierte a microsegundos multiplicándolo por 1000 antes de pasarlo a la función base. De manera análoga, la función `delay_s` permite generar retardos en segundos (Figura 25). Para ello, el valor ingresado se multiplica por 1 000 000, convirtiéndolo a microsegundos, y luego se pasa a la función `esp_rom_delay_us`.

```
void delay_ms(uint32_t ms)
{
    esp_rom_delay_us(ms * 1000); //1ms = 1000microsS
}
```

Figura 24: Implementación de la función `delay_ms` utilizando `esp_rom_delay_us`

```
void delay_s(uint32_t s)
{
    esp_rom_delay_us(s * 1000000); //1s= 1000000 microsegundos
}
```

Figura 25: Implementación de la función `delay_s` utilizando `esp_rom_delay_us`

El archivo `delay.h` contiene las declaraciones de las funciones implementadas, permitiendo su utilización desde otros módulos del programa (Figura 26).

```
1  #ifndef DELAY_H
2  #define DELAY_H
3
4  #include <stdint.h>
5
6  void delay_ms(uint32_t ms);
7  void delay_s(uint32_t s);
8
9  #endif
```

Figura 26: Definición de las funciones de retardo.

3.4 Implementación del parpadeo

Referencias

- Clangd. (s.f.). *Features*. Consultado el 10 de abril de 2026 de <https://clangd.lvm.org/features#fixes>
- Espressif Systems. (s. f.). *Application startup flow*. Consultado el 10 de abril de 2026 de <https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-guides/startup.html>
- Espressif Systems. (s. f.). *build system*. Consultado el 10 de abril de 2026 de <https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-guides/build-system.html>
- Espressif Systems. (s. f.). *Configuration structure*. Consultado el 10 de abril de 2026 de https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-guides/kconfig/configuration_structure.html#sdkconfig-file
- Espressif Systems. (s. f.). *Logging library*. Consultado el 10 de abril de 2026 de <https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/system/log.html>
- Espressif Systems. (s. f.). *Partition tables*. Consultado el 10 de abril de 2026 de <https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-guides/partition-tables.html>
- Git. (s.f.). *gitignore*. Consultado el 10 de abril de 2026 de <https://git-scm.com/docs/gitignore>
- GNU Binutils. (s. f.). *Using as*. Consultado el 10 de abril de 2026 de <https://sourceware.org/binutils/docs/as.html#bss>
- Microchip. (s. f.). *7.3.5 Map File*. Consultado el 10 de abril de 2026 de <https://onlinedocs.microchip.com/>
- Microsoft. (2023). *Using Clang in Visual Studio Code*. Visual Studio Core. Consultado el 10 de abril de 2026 de <https://code.visualstudio.com/docs/cpp/config-clang-mac>
- Microsoft. (2025). *C/C++ extension settings*. Visual Studio Core. Consultado el 10 de abril de 2026 de <https://code.visualstudio.com/docs/cpp/customize-cpp-settings>
- Microsoft. (2026). *Developing inside a container*. Visual Studio Core. Consultado el 10 de abril de 2026 de <https://code.visualstudio.com/docs/devcontainers/containers>
- Microsoft. (2026). *User and workspace settings*. Visual Studio Core. Consultado el 10 de abril de 2026 de <https://code.visualstudio.com/docs/configure/settings>
- Espressif Systems. (s.f.). *Peripherals migration guide (ESP-IDF release 5.0)*. Consultado el 21 de Abril de 2026 de <https://docs.espressif.com/projects/esp-idf/en/stable/esp32/migration-guides/release-5.x/5.0/peripherals.html>

